

VOX COMMANDER

Voice Activated Note Commander

Project Title	VOX COMMANDER – Voice Activated Note Commander
Submitted By	Jui Sudhir Tawde
Course	React JS
Academic Year	2025–2026
Batch	BTECH-CSE
Institution	ITM Skills University Kharghar

3.2 Student Details

Name	Jui Sudhir Tawde
Roll Number	150096725149
Class / Division	Sam Altman
Batch	BTECH-CSE
Academic Year	2025–2026
Institution	ITM Skills University Kharghar

3.3 Problem Statement

Traditional note-taking applications rely heavily on manual typing, making them inconvenient during multitasking, meetings, lectures, or situations where hands-free interaction is preferred. Users require a faster, more accessible, and efficient way to create and manage notes using natural speech.

The objective of this project is to develop a voice-controlled note-taking application that enables users to:

- Dictate notes using natural speech
- Execute commands using voice input
- Visualize audio activity in real-time
- Monitor speech analytics and confidence scores
- Export content in multiple formats (PDF, TXT)
- Review session performance through a summary dashboard

The system leverages modern browser APIs and ReactJS to provide an intuitive and interactive note management experience.

3.4 Technology Stack

Frontend Technologies	React JS, JavaScript (ES6+), HTML5, CSS3
Browser APIs	Web Speech API, Web Audio API, Speech Synthesis API, Local Storage API
React Concepts	Functional Components, React Hooks, Custom Hooks, State Management, Reusable Components
Libraries	Lucide React Icons, jsPDF

Version Control	Git, GitHub
Deployment	Vercel

3.5 Component Architecture

The application follows a modular React architecture using reusable components, custom hooks, services, and utility modules.

System Architecture Flow

(1) User Voice Input
(2) useSpeechRecognition Hook
(3) processTranscriptCommand Service
(4) Note Editor / Command Execution
(5) Statistics & Telemetry Tracking
(6) Session Summary Dashboard
(7) Export Features

Components

Component	Description
Header	Controls navigation, theme switching, and export actions.
MicrophoneToggleBar	Controls for starting, stopping, muting, and resuming speech recognition.
NoteTitleEditor	Allows users to create and edit the note title.
TextOutputEditor	Displays dictated content and supports manual note editing.
StatisticsPanel	Shows real-time statistics: characters, paragraphs, confidence score, duration.
CommandRegistry	Displays the list of supported voice commands.
TelemetryLog	Records and displays system activities, command executions, and events.
SessionSummaryPage	Displays complete session analytics and export options.

VoiceSpectrogram	Real-time microphone visualization using frequency analysis.
------------------	--

Custom Hooks

Hook	Responsibility
useSpeechRecognition()	Manages speech recognition, transcript generation, confidence tracking, auto-restart, and muting.
useAudioAnalyser()	Handles microphone audio analysis and supplies frequency data for visualization.
useLocalStorage()	Provides persistent storage for notes, themes, and user preferences.
useTelemetry()	Manages activity logs, command tracking, and telemetry records.

Services

Service File	Responsibility
commandParser.js	Processes voice commands and performs corresponding note actions.
speechFeedback.js	Provides spoken confirmations and feedback using speech synthesis.
exportPDF.js	Generates downloadable PDF files of notes.
exportHelpers.js	Handles TXT exports and telemetry log downloads.

Utility Module

statistics.js — Calculates and prepares session analytics and summary statistics.

3.5.1 Project Folder Structure

```
src/
├── components/
│   ├── Header.jsx
│   ├── MicrophoneToggleBar.jsx
│   ├── NoteTitleEditor.jsx
│   ├── TextOutputEditor.jsx
│   ├── StatisticsPanel.jsx
│   └── CommandRegistry.jsx
```

```
■ ■■■ TelemetryLog.jsx
■ ■■■ SessionSummaryPage.jsx
■ ■■■ VoiceSpectrogram.jsx
■
■■■ hooks/
■ ■■■ useSpeechRecognition.js
■ ■■■ useLocalStorage.js
■ ■■■ useTelemetry.js
■ ■■■ useAudioAnalyser.js
■
■■■ services/
■ ■■■ commandParser.js
■ ■■■ speechFeedback.js
■ ■■■ exportPDF.js
■ ■■■ exportHelpers.js
■
■■■ utils/
■ ■■■ statistics.js
■
■■■ App.jsx
■■■ App.css
■■■ main.jsx
■■■ index.css
```

3.6 Features

Voice Recognition Features

- Continuous Speech Recognition
- Real-Time Voice Dictation
- Interim Transcript Display
- Final Transcript Processing
- Confidence Score Tracking
- Automatic Engine Restart
- Pause and Resume Listening

Smart Note Management

- Voice-Based Note Creation
- Manual Text Editing
- Editable Note Title
- Auto-Save Functionality

- Local Storage Persistence

Audio Visualization

- Live Voice Spectrogram
- Frequency Analysis
- Real-Time Audio Monitoring
- Web Audio API Integration

Statistics Dashboard

- Character Counter
- Paragraph Counter
- Confidence Meter
- Speech Duration Tracker

Session Summary Dashboard

- Total Words Dictated
- Commands Executed
- Characters Written
- Session Duration
- Recognition Confidence
- Error Statistics

Telemetry Monitoring

- System Activity Logs
- Command Execution Logs
- Speech Recognition Logs
- Downloadable Log Reports

Export Features

- Export Notes as PDF
- Export Notes as TXT
- Download Telemetry Logs

User Experience Features

- Dark Theme
- Light Theme
- Voice Feedback
- Responsive Design

- Modular Interface

3.7 Screenshots

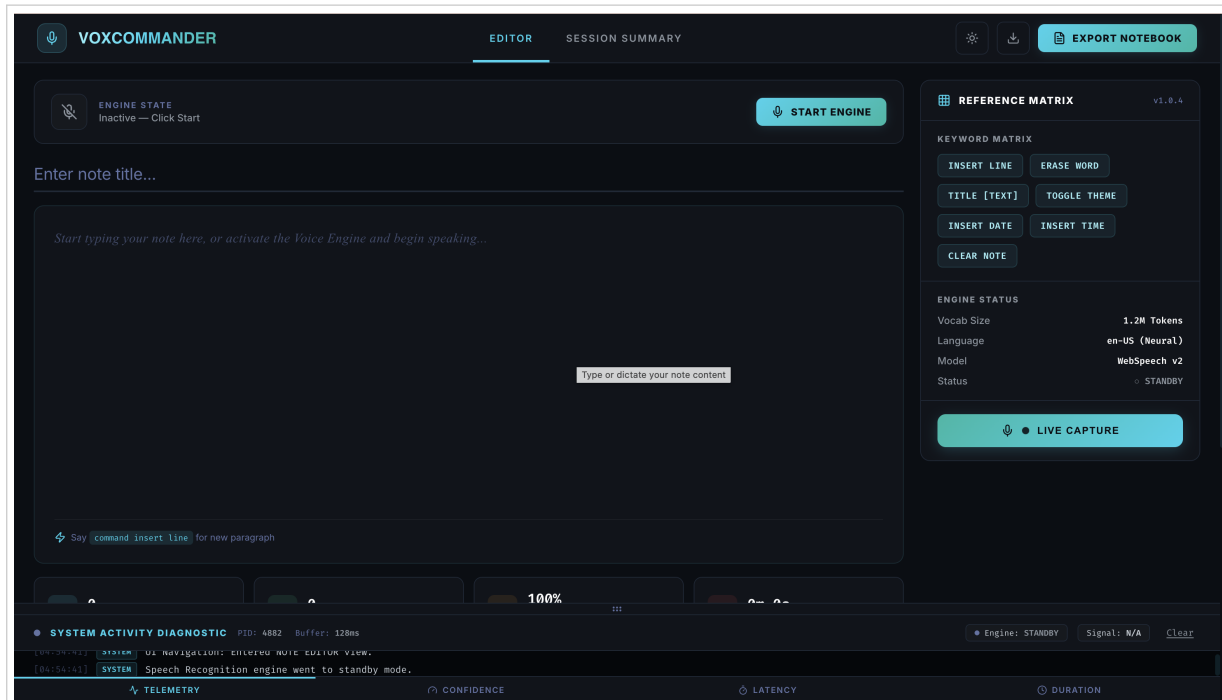


Figure 1: Main Editor View (Dark Theme) — engine in standby state with note editor and keyword command reference

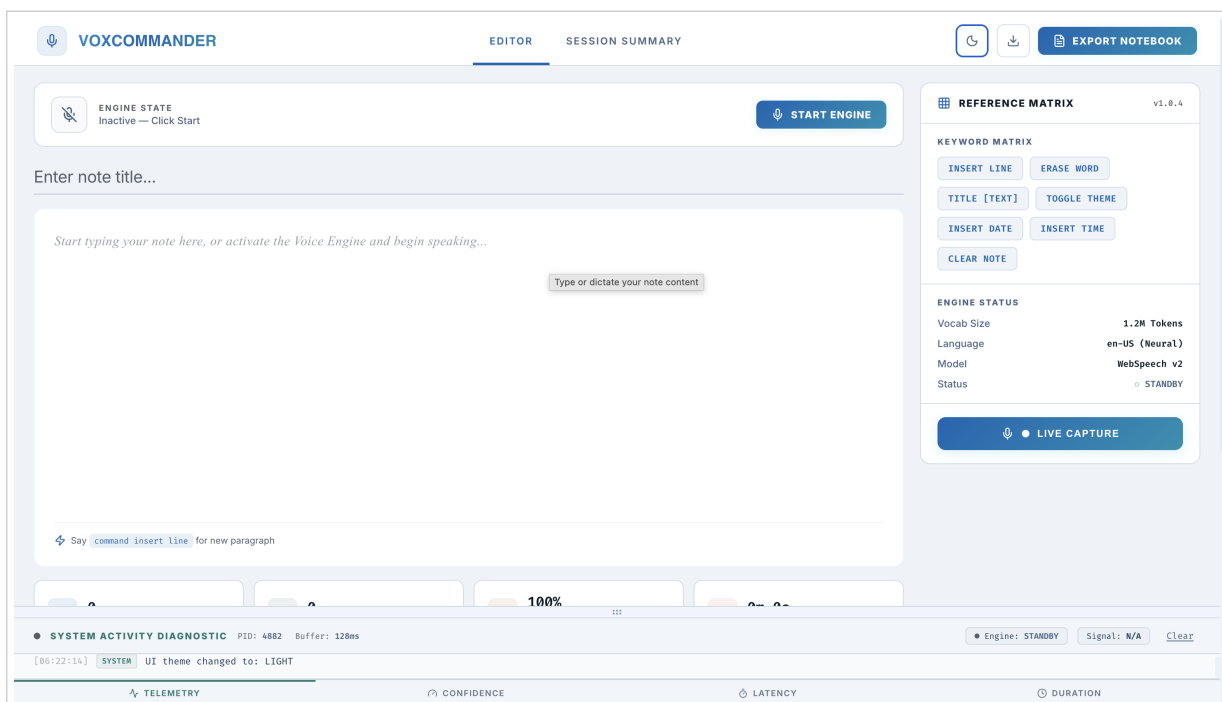


Figure 2: Main Editor View (Light Theme) — demonstrating the theme toggle feature

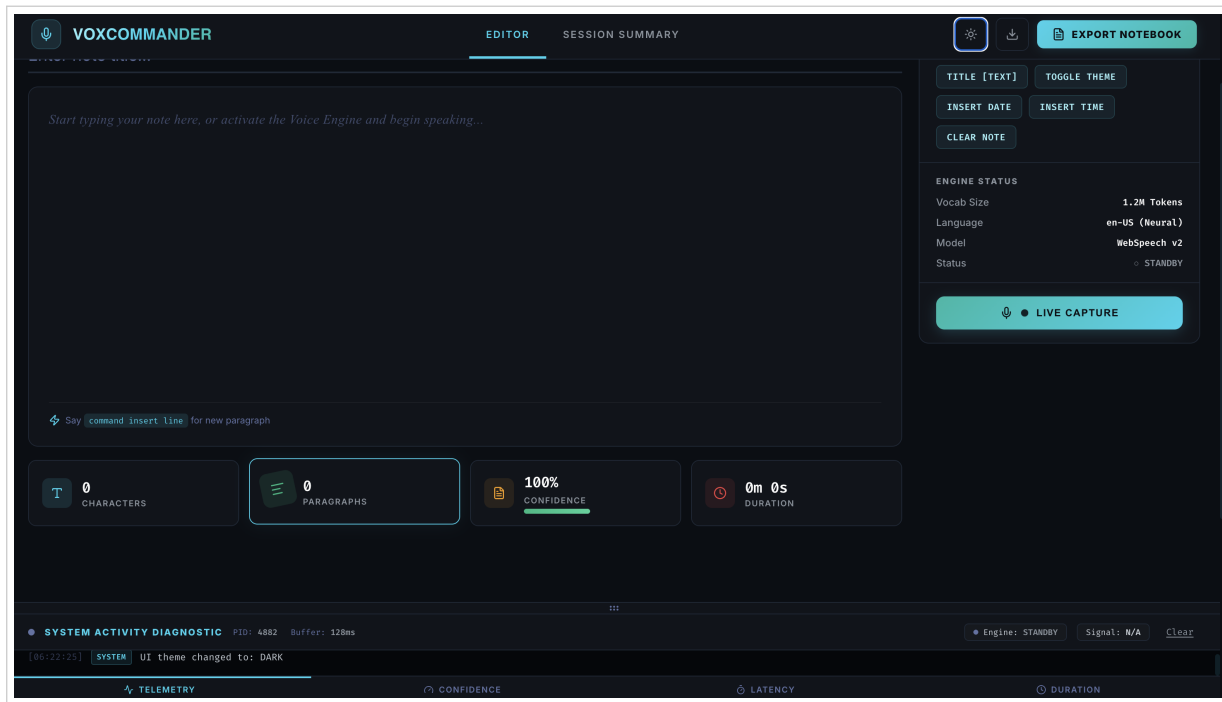


Figure 3: Statistics Panel — Characters, Paragraphs, Confidence (100%) and Duration counters

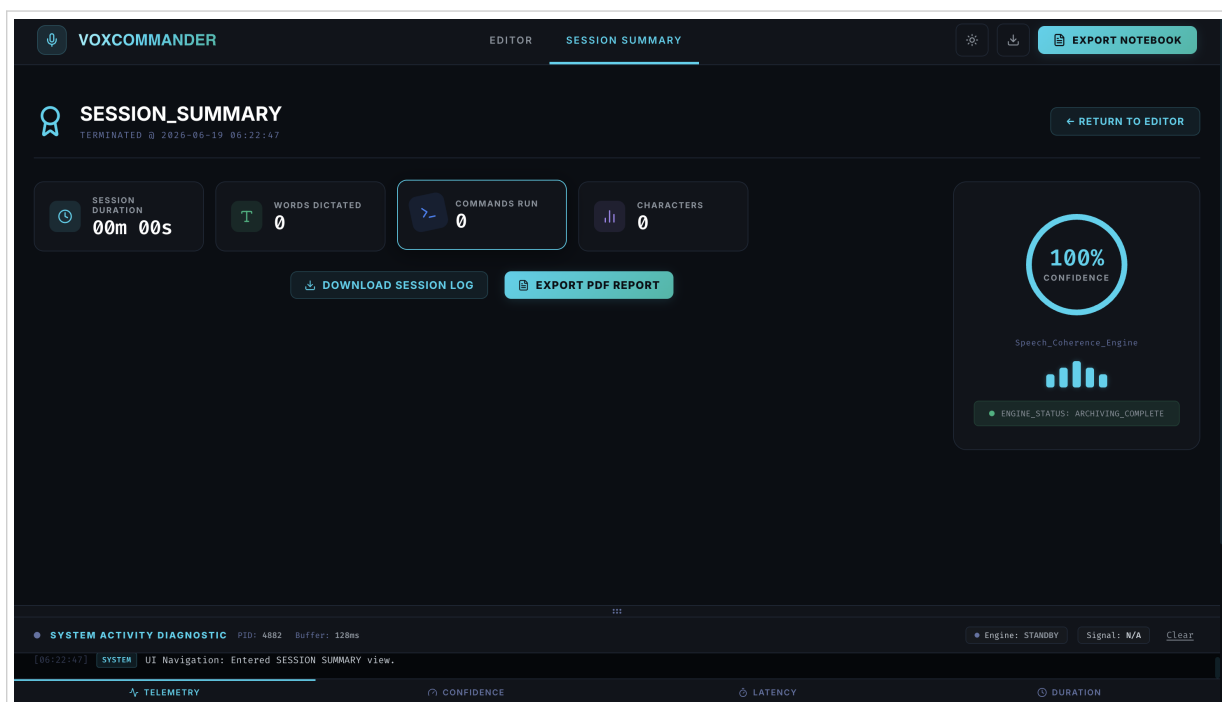


Figure 4: Session Summary Page (Dark Theme) — session metrics and export options

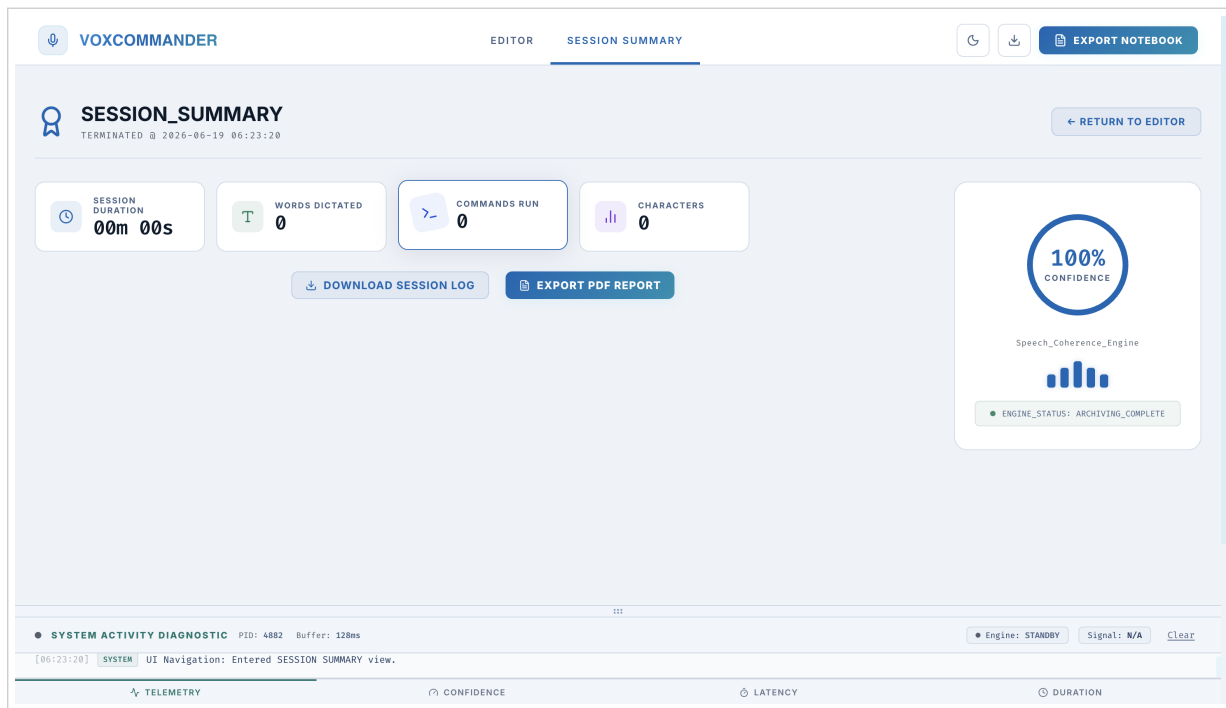


Figure 5: Session Summary Page (Light Theme) — same analytics with Speech Coherence Engine status

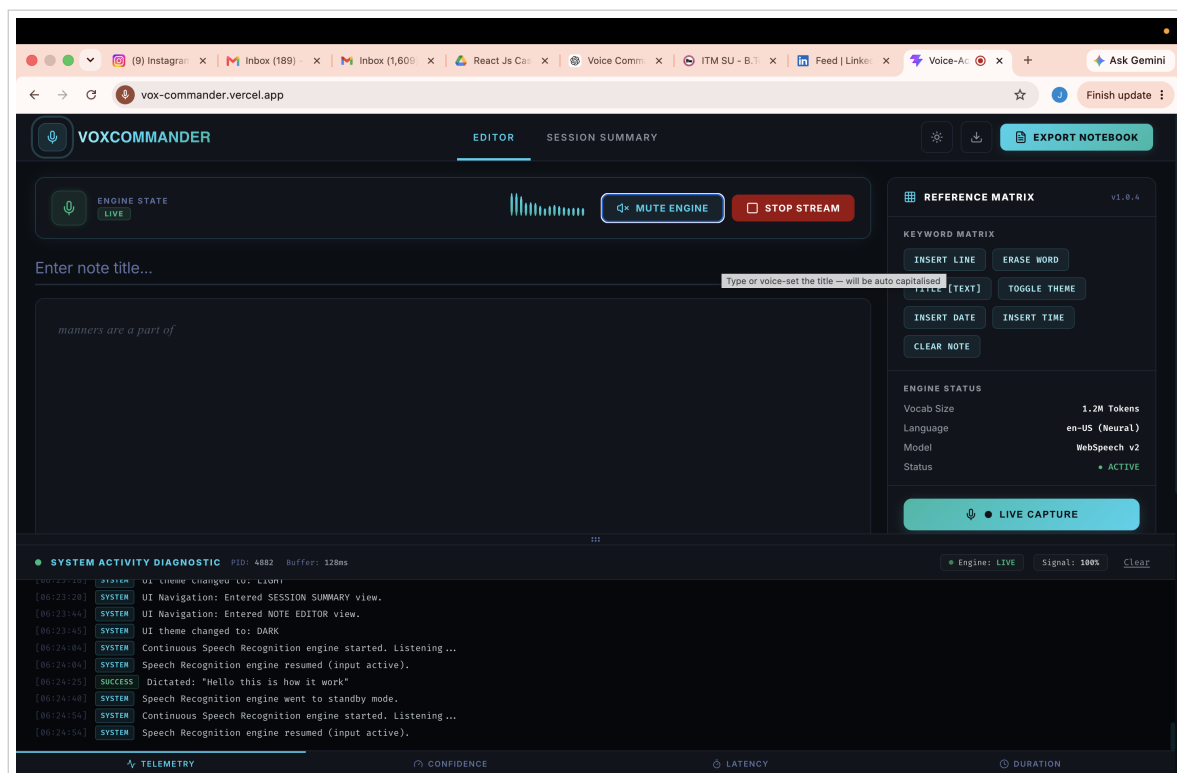


Figure 6: Live Voice Dictation — engine LIVE with audio spectrogram and active telemetry log

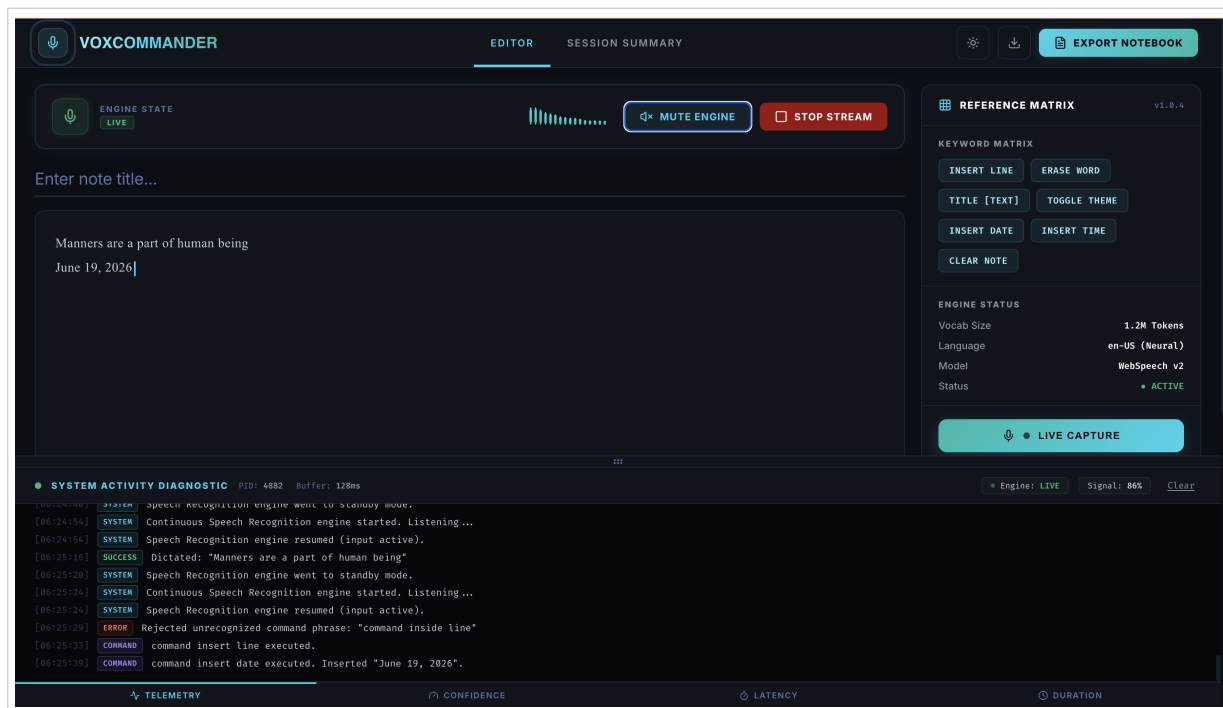


Figure 7: Dictated content with voice commands executed (insert line, insert date)

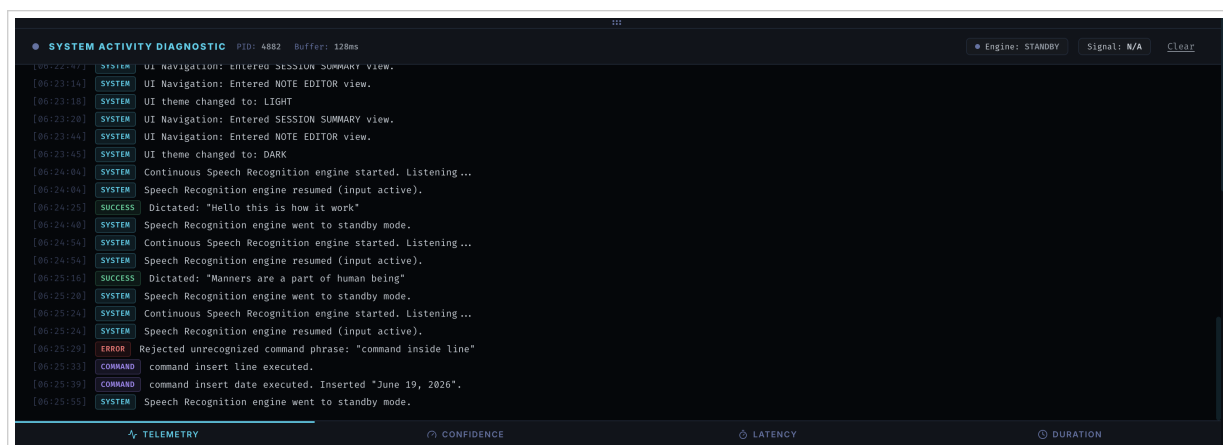


Figure 8: Full Telemetry Log — system activity diagnostic with SUCCESS, ERROR, and COMMAND entries

3.8 GitHub Repository

The complete source code is publicly available on GitHub:

https://github.com/juitawde/vox_commander.git

3.9 Live Demo

The application is deployed and accessible at:

<https://vox-commander.vercel.app/>

3.10 Conclusion

VOX COMMANDER is a modern voice-controlled note-taking application developed using React JS and browser-based speech technologies. The application successfully integrates speech recognition, voice commands, audio visualization, telemetry tracking, session analytics, and export functionalities into a unified platform.

The project demonstrates practical implementation of React Hooks, Custom Hooks, Web Speech API, Web Audio API, Speech Synthesis API, Local Storage, and component-based software architecture. Through hands-free interaction and real-time analytics, the application enhances productivity while providing an accessible and engaging user experience.

The project serves as a comprehensive demonstration of modern frontend development principles and browser API integration using ReactJS.